

MODULE 23

Bidirectional RNN — In-Depth Intuition

Lectures Covered

Lecture 1 — Introduction to Bidirectional RNN

Lecture 2 — Working of Bidirectional RNN

Lecture 1: Introduction to Bidirectional RNN

A standard RNN reads a sequence from left to right — each hidden state only has access to past tokens. A Bidirectional RNN (BiRNN) runs two separate RNNs simultaneously: one forward through the sequence, one backward. At each position, both hidden states are combined — giving the model full context from both directions.

The Core Limitation of Unidirectional RNNs

Consider the sentence:

Context problem

'The bank can guarantee deposits will eventually cover future tuition costs.'
'He sat by the bank of the river and watched the fish.'

To correctly interpret 'bank', the model needs words that come AFTER it.
A left-to-right RNN processing 'The bank...' has no access to 'river' yet.
A Bidirectional RNN sees the full sentence from both directions simultaneously.

What is a Bidirectional RNN?

- Two RNN layers process the same sequence: one forward (left to right), one backward (right to left).
- At each time step *t*, both hidden states are available: *h_forward_t* and *h_backward_t*.
- These are concatenated (or summed/averaged) to form the final output at step *t*.
- The model captures both past context (what came before) and future context (what comes after).

Human reading analogy

When you proofread, you read a sentence twice — forward to understand flow, backward to catch errors and verify grammatical agreement.
A BiRNN does the same: the forward pass captures narrative flow, the backward pass provides global context about where the sentence ends up.

When to Use Bidirectional RNNs

Task	Why BiRNN Helps Suitable?
Named Entity Recognition (NER)	Word type depends on surrounding words Yes
Sentiment Analysis	Negation ('not great') needs full sentence Yes
Part-of-Speech (POS) Tagging	Tag depends on both left and right context Yes
Machine Translation (encoding)	Encoder benefits from full source context Yes
Text Generation (LLM decoding)	Future tokens unknown at generation time No
Speech Recognition (streaming)	Future audio unavailable in real-time Limited

Key constraint

BiRNNs require the FULL sequence to be available before processing begins.
This makes them unsuitable for real-time / autoregressive generation tasks where the model must predict the next token before seeing future tokens.
BERT uses bidirectional attention (similar idea). GPT uses unidirectional.

BiRNN vs Unidirectional RNN — Comparison

Property	Unidirectional RNN	Bidirectional RNN
Context available	Past only (left context)	Past + Future (full context)
Parameters	1x RNN params	2x RNN params (two layers)
Output dimension	hidden_size	2 x hidden_size (concatenated)
Requires full seq	No — can stream	Yes — full sequence needed
Best for	Generation, streaming, LM	Classification, tagging, encoding
Real-world example	GPT (decoder-only)	BERT (encoder-only)

Lecture 2: Working of Bidirectional RNN

This lecture covers the exact computation inside a BiRNN — how the two hidden states are computed, combined, and used for downstream tasks.

Forward and Backward Pass Equations

At each time step t in a sequence of length T :

```
# — FORWARD RNN (left to right) —————
# h_fwd_t = tanh( W_fwd_h * h_fwd_(t-1) + W_fwd_x * x_t + b_fwd )
#
# Processes: x_1 -> x_2 -> x_3 -> ... -> x_T
# h_fwd_t encodes everything seen UP TO position t

# — BACKWARD RNN (right to left) —————
# h_bwd_t = tanh( W_bwd_h * h_bwd_(t+1) + W_bwd_x * x_t + b_bwd )
#
# Processes: x_T -> x_(T-1) -> ... -> x_1
# h_bwd_t encodes everything seen FROM position t ONWARDS

# — COMBINED OUTPUT at each step —————
# y_t = concat( h_fwd_t, h_bwd_t ) # most common: concatenation
# y_t = h_fwd_t + h_bwd_t         # alternative: sum
# y_t = (h_fwd_t + h_bwd_t) / 2   # alternative: average
#
# Concatenation: output dim = 2 * hidden_size
# Sum/Average:  output dim = hidden_size
```

Step-by-Step Example

Sentence: "I love deep learning models" (5 tokens, indices 1 to 5)

Step	Forward h_fwd_t sees...	Backward h_bwd_t sees...	Combined y_t context
t=1 (I)	I	I, love, deep, learning, models	Full sentence from 'I'
t=2 (love)	I, love	love, deep, learning, models	Full sentence from 'love'
t=3 (deep)	I, love, deep	deep, learning, models	Full sentence from 'deep'
t=4 (learning)	I, love, deep, learning	learning, models	Full sentence from 'learning'
t=5 (models)	I, love, deep, learning, models	models	Full sentence from 'models'

Key insight

At every position t , the combined output y_t has access to the ENTIRE sentence. This is fundamentally impossible for a unidirectional RNN at early positions. This full-context representation is why BERT (built on BiLSTM-inspired attention) dramatically outperformed GPT-style left-to-right models on NLU tasks in 2018.

Bidirectional RNN in Keras

```

from tensorflow.keras.layers import Bidirectional, LSTM, GRU, SimpleRNN
from tensorflow.keras.layers import Embedding, Dense
from tensorflow.keras.models import Sequential

# — BiLSTM for sequence classification —————
model = Sequential([
    Embedding(vocab_size, 64, input_length=seq_len),

    # Bidirectional wraps any RNN layer
    # Default merge_mode='concat' -> output dim = 2 * 64 = 128
    Bidirectional(LSTM(64, return_sequences=False)),

    Dense(1, activation='sigmoid') # binary classification
])

# — Merge modes —————
# merge_mode='concat' -> [h_fwd; h_bwd] dim: 2 * units (default)
# merge_mode='sum' -> h_fwd + h_bwd dim: units
# merge_mode='mul' -> h_fwd * h_bwd dim: units
# merge_mode='ave' -> mean(h_fwd, h_bwd) dim: units

# — Stacked BiLSTM for sequence labelling (e.g. NER) —————
model_ner = Sequential([
    Embedding(vocab_size, 64, input_length=seq_len),
    Bidirectional(LSTM(64, return_sequences=True)), # output at every step
    Bidirectional(LSTM(32, return_sequences=True)), # stacked BiLSTM
    Dense(num_tags, activation='softmax') # tag each token
])

# — BiGRU — lighter alternative —————
model_gru = Sequential([
    Embedding(vocab_size, 64, input_length=seq_len),
    Bidirectional(GRU(64)),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

```

Parameter Count — Bidirectional vs Unidirectional

Model	Parameters Notes
SimpleRNN(64)	~4K params Baseline
Bidirectional(SimpleRNN(64))	~8K params 2x — two separate RNN layers
LSTM(64)	~33K params 3 gates + cell state
Bidirectional(LSTM(64))	~66K params 2x LSTM params
GRU(64)	~25K params 2 gates, no separate cell state
Bidirectional(GRU(64))	~50K params 2x GRU params

Practical Tips

- Use `return_sequences=True` inside `Bidirectional` for NER / POS tagging — need output at every step.
- Use `return_sequences=False` for classification — only need the final combined state.
- BiGRU is a good default: similar accuracy to BiLSTM, ~25% fewer parameters.
- `merge_mode='concat'` (default) preserves the most information; preferred in most tasks.

BiRNN in modern NLP context

BERT (2018) is essentially a deep stack of bidirectional self-attention layers — the Transformer successor to BiLSTM encoders.
ELMo (2018) used stacked BiLSTMs to create contextual word embeddings.
Understanding BiRNN is the direct foundation for understanding BERT's encoder.

End of Module 23 Notes — Generative AI Course